Basic clickjacking with CSRF token protection

Basic Clickjacking with CSRF Token Protection

PortSwigger Web Security Academy | Apprentice LabAuthor: Leevan Michael Vaz | **Category:** Clickjacking / UI Redressing | **Date:** July 2026 | **Status:** ✓ Solved

Key Insight

CSRF tokens protect against automated forged requests — they do nothing to stop clickjacking. When a victim clicks a hidden iframe button, they're making a fully legitimate, authenticated request. The CSRF token is valid. The session is real. The protection is bypassed entirely.

Lab Overview

Field	Detail
Target functionality	Delete account (protected by CSRF token)
Objective	Trick victim into deleting their account
Credentials	`wiener : peter`
Victim browser	Google Chrome
Missing headers	`X-Frame-Options`, `CSP: frame-ancestors`

What is Clickjacking?

Clickjacking (also called UI Redressing) is a web attack where an attacker overlays an invisible, interactive element on top of a visible decoy element. The victim

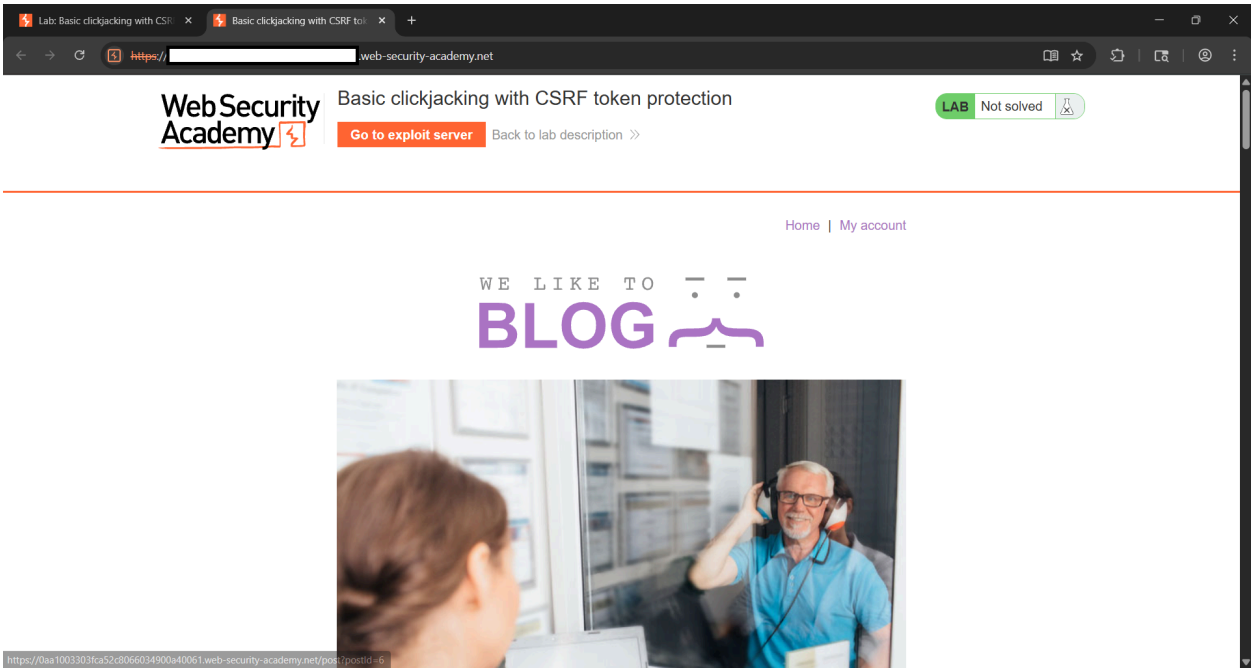
believes they are clicking something harmless — a CAPTCHA, a button, a prize claim — but their click is actually landing on a hidden target underneath.

The critical thing to understand: **clickjacking is not CSRF**. CSRF forges a request on the victim's behalf without their interaction. Clickjacking makes the victim perform the action themselves, unknowingly. This distinction is exactly why CSRF tokens fail here — the request that gets sent is 100% authentic.

Step-by-Step Exploitation

Step 1 — Access the lab and explore the target

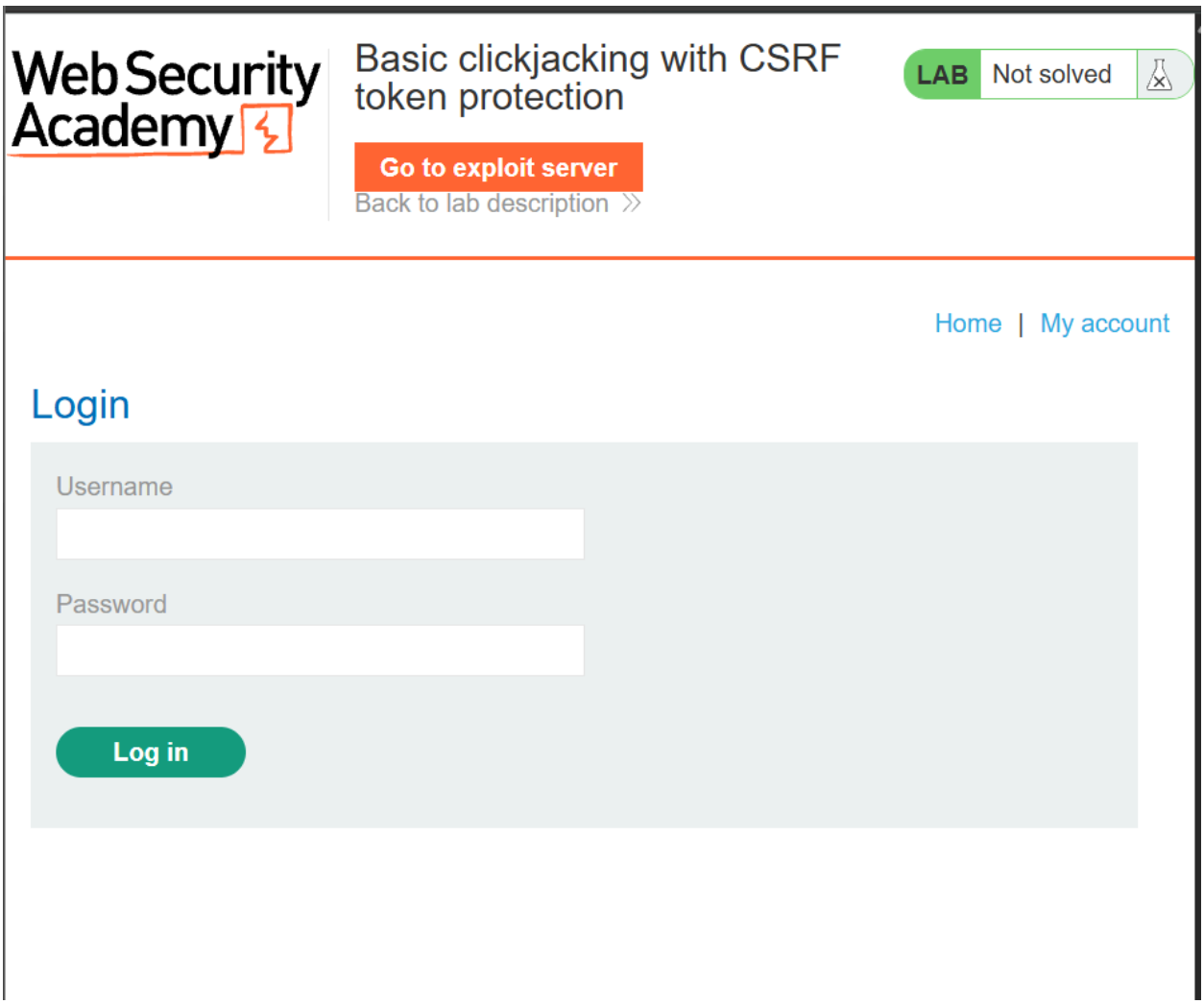
Navigate to the lab URL and observe the target application — a simple blog platform called "We like to blog." The lab header shows a "Go to exploit server" button, which is where we'll host our malicious HTML.



The application has a `/my-account` page accessible after login. This page contains the "Delete account" button that is our target. Crucially, this page has no `X-Frame-Options` or `Content-Security-Policy: frame-ancestors` header in its HTTP response, meaning any external website can embed it in an iframe — the root cause of this vulnerability.

Step 2 — Log in to the application

Use the provided credentials `wiener:peter` on the login page. The lab status shows "Not solved" at this point.



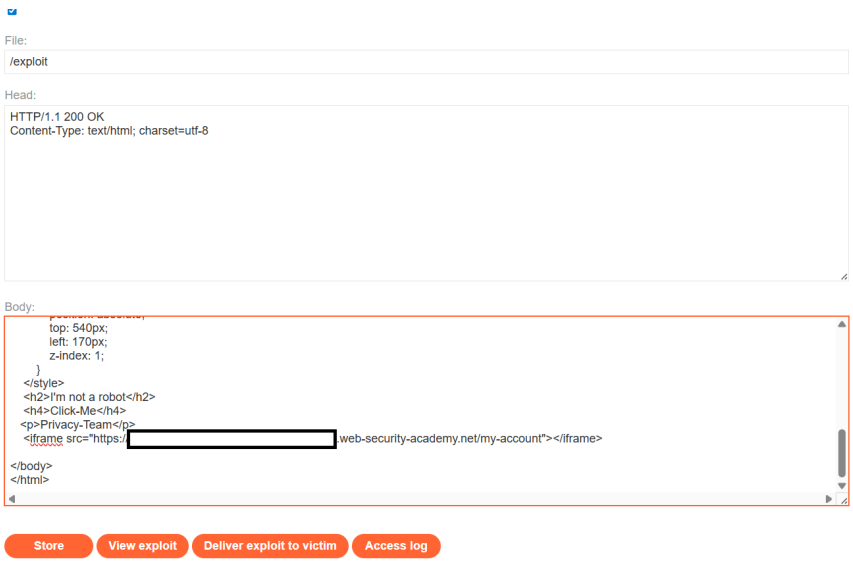
Once logged in, navigate to `/my-account` and note the position of the "Delete account" button on the page. This pixel position is what we need to align our decoy text with in the next step.

Step 3 — Craft the malicious HTML payload

Navigate to the exploit server and paste the following HTML into the Body section. Replace `YOUR-LAB-ID` with your actual unique lab identifier from the URL.

```
<!DOCTYPE html>
<html>
<head>
<style>
    iframe {
        position: relative;
        width: 1135px;
        height: 600px;
        opacity: 0.000001;   /* near-invisible to the victim
*/
        z-index: 2;           /* sits on top — receives all c
licks */
    }
    h2 {
        position: absolute;
        top: 445px;
        left: 60px;
        z-index: 1;           /* below the iframe — victim se
es this */
    }
    h4 {
        position: absolute;
        top: 500px;
        left: 60px;
        z-index: 1;
    }
    p {
        position: absolute;
        top: 540px;
        left: 170px;
        z-index: 1;
    }
</style>
</head>
<h2>I'm not a robot</h2>
```

```
<h4>Click-Me</h4>
<p>Privacy-Team</p>
<iframe src="https://YOUR-LAB-ID.web-security-academy.net/my-account"></iframe>
</body>
</html>
```



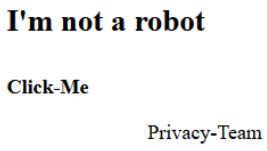
Breaking down each CSS property:

The `iframe` is given `position: relative` so it sits in the normal document flow. The `width` and `height` are set large enough to fully cover the account page including the Delete button. The `opacity: 0.000001` makes the iframe essentially invisible to the human eye — but critically, not `opacity: 0`, because some browsers disable pointer events on fully invisible elements; `0.000001` is imperceptible yet still registers clicks. The `z-index: 2` places the iframe on top of the decoy content, meaning all mouse clicks land on the iframe first.

The `h2`, `h4`, and `p` tags — our decoy content — use `position: absolute` with `top` and `left` values carefully chosen to sit directly over where the Delete Account button appears in the loaded iframe. Their `z-index: 1` places them visually on top but below the iframe in terms of click priority. The victim sees these elements but clicks pass through to the iframe above them.

Step 4 — Align and test the exploit

Click **View exploit** to preview what the victim will see. The decoy page shows only "I'm not a robot", "Click-Me", and "Privacy-Team" — mimicking a CAPTCHA verification screen.



During testing, temporarily set `opacity: 0.5` to make the iframe semi-transparent and verify the "Click-Me" text lines up with the Delete Account button. When you hover over "Click-Me" and the cursor changes to a pointer hand, the alignment is correct. Do not actually click at this stage — clicking Delete Account on your own account during testing will break the lab and require a 20-minute reset.

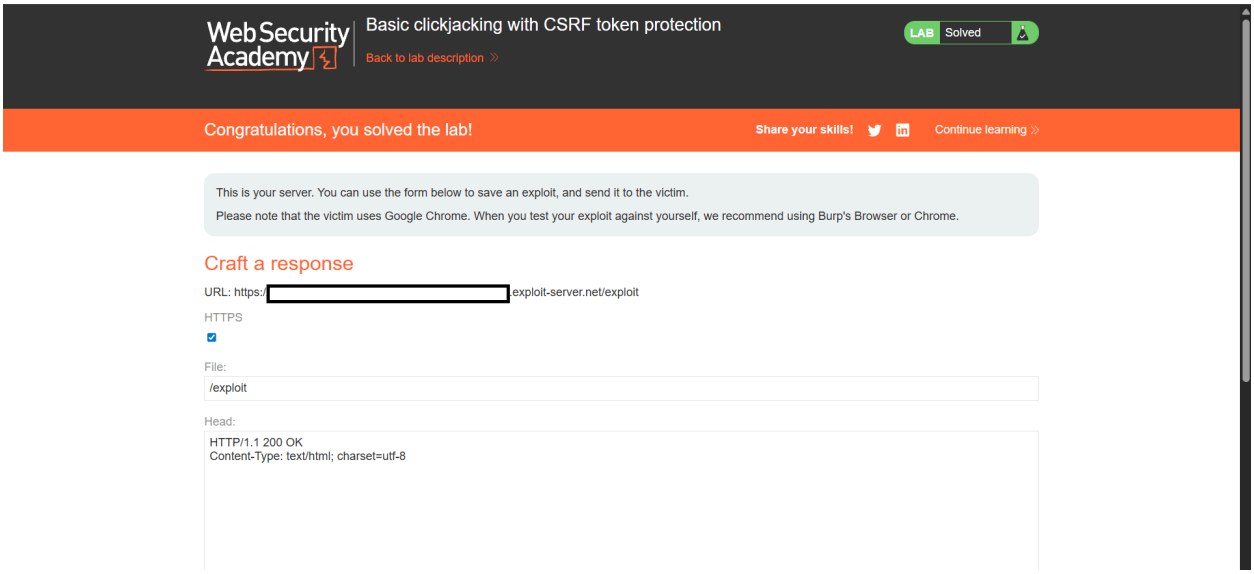
Once alignment is confirmed, set opacity back to `0.000001` and click **Store**.

Step 5 — Deliver the exploit to the victim

Click **Deliver exploit to victim**. The lab system simulates a victim user visiting your exploit page. Their browser loads your HTML, renders the invisible iframe pointing

to their authenticated `/my-account` page, and when they click "Click-Me" — believing it's a CAPTCHA — their click lands on the Delete Account button inside the hidden iframe.

The browser sends the delete request with the victim's real session cookie and the real CSRF token pulled from the form. The server sees a perfectly legitimate authenticated request and deletes the account.



How the Attack Works — The Layer Trick

The browser renders elements in stack order defined by `z-index`. Higher values appear on top visually and receive pointer events first.

```
z-index: 2  →  iframe (topmost layer — receives all clicks, invisible to victim)
z-index: 1  →  decoy text (victim sees this, but clicks pass through)
(default)   →  page background
```

The victim's eyes see the z-index 1 layer. Their mouse clicks land on the z-index 2 layer. These are two different things, and that gap is the entire attack.

Why CSRF tokens don't help here:

In a standard CSRF attack, the attacker crafts a forged request from their own server. The server rejects it because the CSRF token is missing or wrong.

In clickjacking, the victim's own browser makes the request. The victim's browser has the session cookie. The victim's browser loads the account page and gets a fresh, valid CSRF token embedded in the form. When the victim clicks, that token is submitted along with the request. Everything checks out. The server has no way to distinguish this from a legitimate intentional click.

```
Standard CSRF (blocked):
Attacker server  →  forged POST /delete  →  server rejects (invalid token)

Clickjacking (succeeds):
Victim browser  →  legitimate POST /delete  →  server accepts (valid token, valid sessio
n)
```

Remediation

X-Frame-Options header — add to all sensitive pages:

```
X-Frame-Options: DENY
```

This prevents any external domain from embedding the page in an iframe. Simple and effective, though it's considered the legacy approach.

Content Security Policy — the modern standard:

```
Content-Security-Policy: frame-ancestors 'none'
```

More flexible than `X-Frame-Options`. You can also allow same-origin framing only with `frame-ancestors 'self'`, or whitelist specific trusted domains explicitly.

Frame-busting JavaScript — not reliable:

```
if (self !== top) {
    top.location = self.location;
}
```

This can be bypassed by an attacker setting the `sandbox` attribute on the iframe, which prevents the embedded page's JavaScript from navigating the parent frame. Never rely on this alone.

The correct defense is always the HTTP response headers — `X-Frame-Options` and `CSP frame-ancestors` together.

Key Takeaways

Vulnerability class	Clickjacking / UI Redressing
Root cause	Missing frame protection headers
CSRF tokens prevent this?	No — they protect against different attack classes
Real-world targets	Account deletion, fund transfers, role/permission changes, settings modification
Correct fix	`X-Frame-Options: DENY` + `CSP: frame-ancestors 'none'`

The most important lesson from this lab: **CSRF tokens and clickjacking protections are orthogonal**. One without the other leaves a gap. Developers implementing CSRF protection often mistakenly believe they've also covered clickjacking — this lab demonstrates exactly why that assumption is wrong.

Written by Leevan Michael Vaz · PortSwigger Web Security Academy · Clickjacking Series · July 2026